

PROJECT MANAGEMENT (WITH GIT)

Programs 3, 4 and 5 - Complete Lab Instructions

1. One-Time Setup Before the Lab

Run these checks once on the system before attempting any of the three programs.

Step 1: Check whether Git is installed

```
git --version
```

Expected: a version such as git version 2.x.x. If Git is not found, install Git first and reopen the terminal.

Step 2: Configure your identity for commits

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
git config --global --list
```

Important: user.name and user.email identify the author of a commit. They are not your GitHub login password.

Step 3: Authentication only when a remote repository requires it

For a public GitHub repository, cloning usually requires no login. For a private repository or for pushing changes, authenticate using the method supported by your Git hosting service. GitHub HTTPS authentication uses a Personal Access Token (PAT) or a credential manager instead of the normal account password.

- Username: your GitHub/GitLab username, when prompted.
- Password field: enter the PAT/token if HTTPS authentication is requested; do not place the token inside lab files or screenshots.
- Alternative: SSH keys can be used if already configured.

Step 4: Useful checks before each experiment

```
pwd          # show current folder (macOS/Linux/Git Bash)
ls           # list files (macOS/Linux/Git Bash)
git status   # check repository state
git branch   # show local branches
git remote -v # show configured remotes
```

PROGRAM 3 - Stash Changes, Switch Branches, and Apply the Stash

Syllabus requirement: Write the commands to stash your changes, switch branches, and then apply the stashed changes.

Aim

To temporarily save uncommitted changes using Git stash, switch to another branch, and then restore the saved changes.

Files used in this demonstration

File	Purpose
notes.txt	Initial file used to create an uncommitted change.
feature.txt	A sample file committed on feature-branch so the branch is visibly different.

Command flow

Initialize → Commit base file → Create feature branch → Return to master → Modify file → STASH → Switch branch → APPLY stash

Step 1: Create a working folder and initialize the repository

```
mkdir Program3_Stash
cd Program3_Stash
git init -b master
```

If your Git version does not support “-b master”, use git init and then git branch -M master.

Step 2: Create and commit the first file

```
echo "Initial project content" > notes.txt
git add notes.txt
git commit -m "Initial commit"
```

Step 3: Create the branch required for switching

```
git switch -c feature-branch
echo "Feature branch content" > feature.txt
git add feature.txt
git commit -m "Add feature file"
```

Alternative command: Older Git versions can use: git checkout -b feature-branch

Step 4: Return to master and make an unfinished change

```
git switch master
echo "Unfinished change" >> notes.txt
git status
```

At this point notes.txt should appear as modified. Do not commit this change; it is intentionally left unfinished.

Step 5: Stash the unfinished change

```
git stash push -m "Temporary changes for Program 3"  
git stash list  
git status
```

After stashing, the working tree becomes clean. The changes are stored in the stash list.

Step 6: Switch to another branch

```
git switch feature-branch  
git branch
```

Step 7: Apply the saved changes

```
git stash list  
git stash apply stash@{0}  
git status  
cat notes.txt
```

apply vs pop: git stash apply restores the changes but keeps the stash entry. git stash pop restores the changes and normally removes that stash entry. The syllabus specifically asks to apply the stashed changes, so git stash apply is the clearest command.

Step 8: Optional cleanup after successful verification

```
git stash drop stash@{0}
```

Expected observation

- Before stash: notes.txt is shown as modified.
- After git stash: working tree is clean and git stash list contains one entry.
- After switching and applying: the modification reappears on feature-branch.

Result

The uncommitted changes were temporarily stored using Git stash, the branch was changed, and the saved changes were successfully restored using git stash apply.

Quick viva

- What is the purpose of git stash?
- What is the difference between git stash apply and git stash pop?
- How do you list all stashed entries?
- Why might Git refuse to switch branches when uncommitted changes exist?

PROGRAM 4 - Clone a Remote Git Repository

Syllabus requirement: Clone a remote Git repository to your local machine.

Aim

To create a complete local copy of an existing remote Git repository using git clone.

Key point before starting

Do not run git init before cloning: git clone already creates the destination directory, copies the repository history, checks out a working branch, and configures the remote named origin. Running git init first is unnecessary.

Files in the supplied practice repository

File	Purpose
README.md	Simple repository description.
app.py	Small Python file to prove the cloned files are available locally.

Method A - Using GitHub/GitLab (normal lab method)

Step 1: Copy the repository URL

Example HTTPS format:

```
https://github.com/<username>/<repository>.git
```

Step 2: Move to the folder where you want the local copy

```
cd <your-parent-folder>
```

Step 3: Clone the repository

```
git clone https://github.com/<username>/<repository>.git
```

Step 4: Enter the cloned folder and verify

```
cd <repository>
git status
git remote -v
git branch
git log --oneline -5
```

Authentication note

- Public repository: clone normally without credentials.
- Private repository: Git may open a browser/credential manager or request username and a token/PAT.
- Do not type your normal account password where the hosting service requires a PAT.

Method B - Supplied offline practice files

The ZIP supplied with this manual includes Program4_Clone/source_repo and an already-tested cloned_repo. You can repeat the clone locally without Internet access:

```
cd Program4_Clone
git clone source_repo my_new_clone
cd my_new_clone
git status
git remote -v
python app.py
```

Expected Python output: Hello from the cloned Git repository

What git clone automatically does

- Creates a new local directory.
- Copies tracked files and commit history.
- Creates the .git repository metadata.
- Adds the source as a remote named origin.
- Checks out the default branch.

Result

The remote repository was cloned successfully to the local system and its files, history, branch information, and origin remote were verified.

Quick viva

- What is the difference between git init and git clone?
- What is origin in Git?
- Does cloning a public repository always require a username and password?
- Which command shows the remote repository URL?

PROGRAM 5 - Fetch Latest Remote Changes and Rebase the Local Branch

Syllabus requirement: Fetch the latest changes from a remote repository and rebase your local branch onto the updated remote branch.

Aim

To download new commits from the remote repository without immediately merging them, inspect the updated history, and then rebase the local commits on top of the latest remote branch.

Important prerequisite

Program 5 must be performed inside a repository that already has a remote named origin. The easiest starting point is a repository created by Program 4 using git clone.

Step 1: Verify current branch and remote

```
git status
git branch --show-current
git remote -v
```

For the syllabus wording, the examples below use master. If your repository uses main, replace master with main in the commands.

Step 2: Make sure you understand the current history

```
git log --oneline --graph --decorate --all -10
```

Step 3: Fetch the latest information from origin

```
git fetch origin
```

git fetch downloads new commits and updates remote-tracking references such as origin/master. It does not automatically modify your working files.

Step 4: Inspect the difference after fetching

```
git status
git log --oneline --graph --decorate --all -10
```

Step 5: Rebase the current local branch onto the updated remote branch

```
git rebase origin/master
```

If your default branch is main: Use: git rebase origin/main

Step 6: Verify the final history

```
git status
git log --oneline --graph --decorate --all -10
```

After a successful rebase, your local commits should appear after the newest remote commits, producing a linear history.

If a rebase conflict occurs

A conflict is possible when both local and remote commits modify the same part of a file. Use the following recovery sequence only if Git reports a conflict:

```
git status
# Open the conflicted file(s), remove conflict markers, and keep the correct content
git add <resolved-file>
git rebase --continue
```

To cancel the rebase and return to the state before rebasing:

```
git rebase --abort
```

Supplied offline practice setup

The ZIP contains a fully tested local simulation with a bare remote repository plus two clones: one simulates another developer and one is the local student repository.

Folder	Meaning
remote.git	Simulated shared remote repository.
upstream_repo	Simulates another developer who pushed a newer remote commit.
local_repo	Student repository containing a local commit that was rebased.

To inspect the tested result:

```
cd Program5_Fetch_Rebase/local_repo
git log --oneline --graph --decorate --all -10
git status
```

Command meaning at a glance

Command	Meaning
git fetch origin	Download remote commits and update origin/* references.
git rebase origin/master	Move/replay local commits on top of the latest origin/master.
git rebase --continue	Continue after resolving a conflict.
git rebase --abort	Cancel the current rebase operation.

Result

The latest remote changes were fetched successfully and the local branch was rebased onto the updated remote branch.

Quick viva

- What is the difference between git fetch and git pull?
- What does origin/master represent?